

376

顺序容器只定义了很少的操作：在多数情况下，我们可以添加和删除元素、访问首尾元素、确定容器是否为空以及获得指向首元素或尾元素之后位置的迭代器。

我们可以想象用户可能还希望做其他很多有用的操作：查找特定元素、替换或删除一个特定值、重排元素顺序等。

标准库并未给每个容器都定义成员函数来实现这些操作，而是定义了一组泛型算法 (generic algorithm)：称它们为“算法”，是因为它们实现了一些经典算法的公共接口，如排序和搜索；称它们是“泛型的”，是因为它们可以用于不同类型的元素和多种容器类型（不仅包括标准库类型，如 `vector` 或 `list`，还包括内置的数组类型），以及我们将看到的，还能用于其他类型的序列。



10.1 概述

大多数算法都定义在头文件 `algorithm` 中。标准库还在头文件 `numeric` 中定义了一组数值泛型算法。

一般情况下，这些算法并不直接操作容器，而是遍历由两个迭代器指定的一个元素范围（参见 9.2.1 节，第 296 页）来进行操作。通常情况下，算法遍历范围，对其中每个元素进行一些处理。例如，假定我们有一个 `int` 的 `vector`，希望知道 `vector` 中是否包含一个特定值。回答这个问题最方便的方法是调用标准库算法 `find`：

```
int val = 42; // 我们将查找的值
// 如果在 vec 中找到想要的元素，则返回结果指向它，否则返回结果为 vec.cend()
auto result = find(vec.cbegin(), vec.cend(), val);
// 报告结果
cout << "The value " << val
      << (result == vec.cend()
          ? " is not present" : " is present") << endl;
```

传递给 `find` 的前两个参数是表示元素范围的迭代器，第三个参数是一个值。`find` 将范围中每个元素与给定值进行比较。它返回指向第一个等于给定值的元素的迭代器。如果范围中无匹配元素，则 `find` 返回第二个参数来表示搜索失败。因此，我们可以通过比较返回值和第二个参数来判断搜索是否成功。我们在输出语句中执行这个检测，其中使用了条件运算符（参见 4.7 节，第 134 页）来报告搜索是否成功。

由于 `find` 操作的是迭代器，因此我们可以用同样的 `find` 函数在任何容器中查找值。例如，可以用 `find` 在一个 `string` 的 `list` 中查找一个给定值：

```
string val = "a value"; // 我们要查找的值
// 此调用在 list 中查找 string 元素
auto result = find(lst.cbegin(), lst.cend(), val);
```

类似的，由于指针就像内置数组上的迭代器一样，我们可以用 `find` 在数组中查找值：

377

```
int ia[] = {27, 210, 12, 47, 109, 83};
int val = 83;
int* result = find(begin(ia), end(ia), val);
```

此例中我们使用了标准库 `begin` 和 `end` 函数（参见 3.5.3 节，第 106 页）来获得指向 `ia` 中首元素和尾元素之后位置的指针，并传递给 `find`。

还可以在序列的子范围中查找，只需将指向子范围首元素和尾元素之后位置的迭代器

(指针) 传递给 `find`。例如, 下面的语句在 `ia[1]`、`ia[2]` 和 `ia[3]` 中查找给定元素:

```
// 在从 ia[1] 开始, 直至 (但不包含) ia[4] 的范围内查找元素
auto result = find(ia + 1, ia + 4, val);
```

算法如何工作

为了弄清这些算法如何用于不同类型的容器, 让我们更近地观察一下 `find`。`find` 的工作是在一个未排序的元素序列中查找一个特定元素。概念上, `find` 应执行如下步骤:

1. 访问序列中的首元素。
2. 比较此元素与我们要查找的值。
3. 如果此元素与我们要查找的值匹配, `find` 返回标识此元素的值。
4. 否则, `find` 前进到下一个元素, 重复执行步骤 2 和 3。
5. 如果到达序列尾, `find` 应停止。
6. 如果 `find` 到达序列末尾, 它应该返回一个指出元素未找到的值。此值和步骤 3 返回的值必须具有相容的类型。

这些步骤都不依赖于容器所保存的元素类型。因此, 只要有一个迭代器可用来访问元素, `find` 就完全不依赖于容器类型 (甚至无须理会保存元素的是不是容器)。

迭代器令算法不依赖于容器, ……

在上述 `find` 函数流程中, 除了第 2 步外, 其他步骤都可以用迭代器操作来实现: 利用迭代器解引用运算符可以实现元素访问; 如果发现匹配元素, `find` 可以返回指向该元素的迭代器; 用迭代器递增运算符可以移动到下一个元素; 尾后迭代器可以用来判断 `find` 是否到达给定序列的末尾; `find` 可以返回尾后迭代器 (参见 9.2.1 节, 第 296 页) 来表示未找到给定元素。

……, 但算法依赖于元素类型的操作

虽然迭代器的使用令算法不依赖于容器类型, 但大多数算法都使用了一个 (或多个) 元素类型上的操作。例如, 在步骤 2 中, `find` 用元素类型的 `==` 运算符完成每个元素与给定值的比较。其他算法可能要求元素类型支持 `<` 运算符。不过, 我们将会看到, 大多数算法提供了一种方法, 允许我们使用自定义的操作来代替默认的运算符。

◀ 378

10.1 节练习

练习 10.1: 头文件 `algorithm` 中定义了一个名为 `count` 的函数, 它类似 `find`, 接受一对迭代器和一个值作为参数。`count` 返回给定值在序列中出现的次数。编写程序, 读取 `int` 序列存入 `vector` 中, 打印有多少个元素的值等于给定值。

练习 10.2: 重做上一题, 但读取 `string` 序列存入 `list` 中。

关键概念: 算法永远不会执行容器的操作

泛型算法本身不会执行容器的操作, 它们只会运行于迭代器之上, 执行迭代器的操作。泛型算法运行于迭代器之上而不会执行容器操作的特性带来了一个令人惊讶但非常必要的编程假定: 算法永远不会改变底层容器的大小。算法可能改变容器中保存的元素

的值，也可能在容器内移动元素，但永远不会直接添加或删除元素。

如我们将在 10.4.1 节（第 358 页）所看到的，标准库定义了一类特殊的迭代器，称为插入器（`inserter`）。与普通迭代器只能遍历所绑定的容器相比，插入器能做更多的事情。当给这类迭代器赋值时，它们会在底层的容器上执行插入操作。因此，当一个算法操作一个这样的迭代器时，迭代器可以完成向容器添加元素的效果，但算法自身永远不会做这样的操作。



10.2 初识泛型算法

标准库提供了超过 100 个算法。幸运的是，与容器类似，这些算法有一致的结构。比起死记硬背全部 100 多个算法，理解此结构可以帮助我们更容易地学习和使用这些算法。在本章中，我们将展示如何使用这些算法，并介绍刻画了这些算法的统一原则。附录 A 按操作方式列出了所有算法。

除了少数例外，标准库算法都对一个范围内的元素进行操作。我们将此元素范围称为“输入范围”。接受输入范围的算法总是使用前两个参数来表示此范围，两个参数分别是指向要处理的第一个元素和尾元素之后位置的迭代器。

虽然大多数算法遍历输入范围的方式相似，但它们使用范围中元素的方式不同。理解算法的最基本的方法就是了解它们是否读取元素、改变元素或是重排元素顺序。



10.2.1 只读算法

379

一些算法只会读取其输入范围内的元素，而从不改变元素。`find` 就是这样一种算法，我们在 10.1 节练习（第 337 页）中使用的 `count` 函数也是如此。

另一个只读算法是 `accumulate`，它定义在头文件 `numeric` 中。`accumulate` 函数接受三个参数，前两个指出了需要求和的元素的范围，第三个参数是和的初值。假定 `vec` 是一个整数序列，则：

```
// 对 vec 中的元素求和，和的初值是 0
int sum = accumulate(vec.cbegin(), vec.cend(), 0);
```

这条语句将 `sum` 设置为 `vec` 中元素的和，和的初值被设置为 0。



`accumulate` 的第三个参数的类型决定了函数中使用哪个加法运算符以及返回值的类型。

算法和元素类型

`accumulate` 将第三个参数作为求和起点，这蕴含着一个编程假定：将元素类型加到和的类型上的操作必须是可行的。即，序列中元素的类型必须与第三个参数匹配，或者能够转换为第三个参数的类型。在上例中，`vec` 中的元素可以是 `int`，或者是 `double`、`long` 或任何其他可以加到 `int` 上的类型。

下面是另一个例子，由于 `string` 定义了 `+` 运算符，所以我们可以调用 `accumulate` 来将 `vector` 中所有 `string` 元素连接起来：

```
string sum = accumulate(v.cbegin(), v.cend(), string(""));
```